



Tutorial: Construcción del Proyecto

"Credenciales Académicas UNQ"

Este documento es una guía paso a paso y detallada para construir el proyecto final de la diplomatura. Está pensado para seguirse de principio a fin, adoptando una arquitectura de **Monorepo** donde el código del contrato inteligente y el de la aplicación web conviven armónicamente.

🔧 Fase 1: Preparación del Entorno (Monorepo)

Un monorepo nos permite manejar el backend (blockchain) y el frontend en un solo lugar, simplificando el control de versiones.

Paso 1.1: Inicialización del Repositorio

Primero, vamos a crear la estructura principal. Abrió tu terminal y ejecutá:

```
# Crea la carpeta principal del proyecto y entra en ella
mkdir unq-credenciales-blockchain
cd unq-credenciales-blockchain

# Inicializa un repositorio de Git vacío
git init
```

Paso 1.2: Inicializar el entorno Blockchain (Foundry)

Vamos a usar Foundry para desarrollar nuestros Smart Contracts. Lo instalaremos en una subcarpeta llamada `blockchain`.

```
# Inicializa un nuevo proyecto de Foundry en la carpeta 'blockchain'
forge init blockchain
```

Tip: Esto creará directorios como `src`, `test` y `script` dentro de la carpeta `blockchain`.

Paso 1.3: Clonar e Integrar el Frontend Starter

Para el frontend, usaremos un template base de Next.js proveído por la cátedra. Lo clonaremos en la carpeta `frontend`, pero borraremos su historial de Git para que sea parte de nuestro monorepo principal.

```
# Asegurate de estar en la raíz del proyecto (unq-credenciales-blockchain)
# Clona el repositorio starter en la carpeta 'frontend'
git clone https://github.com/dpetrocelli/diplo-unq-blockchain-tp-final-starter
frontend

# Elimina el historial de git del starter
rm -rf frontend/.git
```

Paso 1.4: Configuración de Git

Es crucial evitar subir archivos pesados o sensibles (como claves privadas) a GitHub.

Crea un archivo llamado `.gitignore` en la **raíz del proyecto** (`unq-credenciales-blockchain/.gitignore`) y agregó lo siguiente:

```
# Node
node_modules/
frontend/.next/

# Foundry
blockchain/out/
blockchain/cache/
blockchain/broadcast/

# Entorno y variables sensibles
.env
```

Finalmente, hacemos el primer commit estructural:

```
git add .
git commit -m "chore: inicialización del monorepo (foundry + frontend starter)"
```

🔧 Fase 2: Desarrollo del Smart Contract

Vamos a trabajar exclusivamente en la carpeta `blockchain` .

Paso 2.1: Instalar Dependencias (OpenZeppelin)

Usaremos los estándares seguros de OpenZeppelin para nuestro token.

```
cd blockchain
# Instala la librería de contratos de OpenZeppelin
forge install openzeppelin/openzeppelin-contracts --no-commit
```

Paso 2.2: Escribir el Contrato Inteligente

Creá o modificá el archivo `blockchain/src/AcademicCredentials.sol` . Aquí está la guía de implementación:

- 1. Importaciones:** Necesitás heredar de `AccessControl` (para manejar roles) y de `ERC721URIStorage` (para manejar el NFT y su metadata).
- 2. Estructura de Datos:** Definí un `struct Credential` que contenga: `nombreEstudiante` , `dni` , `carrera` , `fechaEmision` y `promedio` .
- 3. Almacenamiento:** Creá un `mapping(uint256 => Credential)` public `credentials`; para guardar la data on-chain.
- 4. Soulbound (Intransferible):** Sobrescribí la función `_update` de ERC721. Si el emisor (`from`) no es la dirección cero (es decir, no es un minteo) y el receptor (`to`) tampoco, debe hacer `revert("Soulbound: non-transferable");` .
- 5. Roles:** Definí un `ISSUER_ROLE` . Asegurate de que la función `issueCredential` tenga el modificador `onlyRole(ISSUER_ROLE)` . El despliegue inicial debe otorgar el `DEFAULT_ADMIN_ROLE` a tu cuenta para poder gestionar a los emisores.

Paso 2.3: Documentación Inicial

Regresá a la raíz del proyecto y creá un `README.md`.

- Redactá una introducción sobre qué hace el proyecto.
- Agregá un diagrama usando Mermaid (``mermaid``) que muestre cómo un emisor carga los datos, el contrato los guarda y el estudiante los verifica.

Fase 3: Pruebas y Seguridad

No desplegamos sin probar. Seguimos en la carpeta `blockchain`.

Paso 3.1: Pruebas Unitarias (Tests)

Abrí `blockchain/test/AcademicCredentials.t.sol` y estructurá tus pruebas:

1. **setUp()**: Desplegá el contrato, asigná roles usando `vm.startPrank` a cuentas de prueba.
2. **Camino Feliz**: Escribí `test_EmitirCredencial()`. Llamá a `issueCredential`, luego buscá la credencial en el `mapping` y usá `assertEq` para validar que los datos guardados son correctos.
3. **Comportamiento Soulbound**: Escribí `test_RevertIf_TransferAttempt()`. Emití un token e intentá usar `transferFrom`. Envolvé esa llamada con `vm.expectRevert("Soulbound: non-transferable");` para asegurar que falle.
4. **Fuzzing**: Escribí `testFuzz_Emision(address estudiante)`. Asegurá que funcione para cualquier dirección válida.

Para correr las pruebas y ver la cobertura:

```
forge test
forge coverage --report lcov
```

(Meta: Lograr +80% de cobertura)

Paso 3.2: Análisis de Seguridad con Slither

Asegurate de tener Python instalado y ejecutá:

```
pip install slither-analyzer
slither .
```

Creá un archivo `SECURITY.md` en la raíz del proyecto, copió las advertencias que tire Slither (si las hay) y escribí una breve mitigación o justificación.

Fase 4: Despliegue en L2 (Base Sepolia)

Paso 4.1: Configuración de Entorno

Dentro de la carpeta `blockchain`, creá un archivo `.env` (¡Asegurate de que no se suba a Git!).

```
PRIVATE_KEY=tu_clave_privada_de_metamask_de_pruebas
BASE_SEPOLIA_RPC_URL=https://sepolia.base.org
```

```
BASESCAN_API_KEY=tu_api_key_de_basescan
```

Paso 4.2: Script de Despliegue

En `blockchain/script/Deploy.s.sol`, escribí un script básico de Foundry que haga `vm.startBroadcast(vm.envUint("PRIVATE_KEY"))`; , despliegue el contrato `AcademicCredentials`, y termine con `vm.stopBroadcast()`.

Paso 4.3: Ejecutar el Deploy y Verificar

```
# Carga las variables de entorno
source .env

# Ejecuta el script, envía la transacción y verifica el contrato en Basescan
forge script script/Deploy.s.sol --rpc-url $BASE_SEPOLIA_RPC_URL --private-key $PRIVATE_KEY --broadcast --verify --etherscan-api-key $BASESCAN_API_KEY
```

¡Guarda la dirección del contrato que te devuelve la terminal! Como paso extra, andá a Basescan, conectá tu billetera y emití 3 credenciales de prueba usando la interfaz de "Write Contract".



Fase 5: Conexión del Frontend

Es hora de darle una interfaz visual. Nos movemos a la carpeta `frontend`.

Paso 5.1: Instalación

```
cd ../frontend
npm install
```

Paso 5.2: Conectar el Contrato

1. **ABI:** Copiá el archivo `AcademicCredentials.json` generado en `blockchain/out/AcademicCredentials.sol/` y pegalo en el frontend (ej. en `src/abi/`).
2. **Dirección:** Encontrá el archivo de constantes del starter (ej. `constants.ts`) y actualizá la variable `CREDENTIALS_ADDRESS` con la dirección que guardaste en el paso 4.3.

Paso 5.3: Configurar Wagmi

En el archivo `wagmi.ts`, asegurate de que la cadena importada y utilizada sea `baseSepolia`.

Paso 5.4: Interfaz Pública (Verificación)

Modificá el componente de lectura. Usá el hook `useReadContract` de Wagmi. Pasale la dirección, el ABI, el nombre de la función (ej. `credentials`) y el `args` (el ID a buscar). Mostrá en pantalla los datos devueltos. Si el token existe, mostrá un .

Paso 5.5: Interfaz del Emisor (Escritura)

En el panel del administrador, vinculá el formulario. Antes de enviar a la blockchain, usá la utilidad `keccak256` y `stringToBytes` de Viem para hashear el Nombre + DNI si es necesario según tu lógica de privacidad, o simplemente enviá los campos al hook `useWriteContract`.

Fase 6: Finalización y Demo

1. Levantá el proyecto localmente:

```
npm run dev
```

2. Probá el flujo completo: conectá tu billetera, emití una credencial como Issuer, y luego buscala en la vista pública.
 3. Grabá un video corto (máximo 5 min) mostrando el funcionamiento.
 4. Subí el video y agregá el enlace al final de tu `README.md`.
-

Opcionales: "El Tablero de Extras"

Si llegaste hasta acá, podés ir por más puntos creando nuevas ramas (`git checkout -b feature/lo-que-sea`):

1. **Deploy en Vercel:** Importá la carpeta `frontend` en Vercel. ¡Tendrás una URL pública real!
2. **Super-Admin UI:** Creá una ruta `/admin` en Next.js donde, usando `useWriteContract`, puedas asignar o revocar el rol de `ISSUER_ROLE` a otras direcciones.
3. **Código QR:** Usá `react-qrcode` en la vista de verificación. Que el QR apunte a `https://tu-app.vercel.app/verify/{ID}`.
4. **Integración IPFS (Pinata):** Creá una API en Next.js que reciba un PDF, lo suba a Pinata, devuelva el CID, y lo guardes como el `tokenURI` al momento de emitir la credencial en la blockchain.

¡Muchos éxitos en la implementación!